

KI-Drohne mit AprilTag-Abwurf

Von der FPV-Renndrohne zur GPS-freien Indoor-Plattform: eigene ArduPilot-Firmware, Raspberry Pi als Companion-Computer, AprilTag-Erkennung auf der Kamera und eine servogesteuerte Nutzlast-Freigabe — mit den Flugversuchen, die dahin geführt haben.

TEAM (4 PERSONEN)

Jannik Höhler · Julian Veerkamp · Sebastian Urmann · Serkan Savranoglu

BETREUUNG

Prof. Dr. Baun

Ziel des Projekts

Ausgangspunkt ist eine flugfertige 3,5"-CineWhoop-FPV-Drohne, die ausschließlich manuell per Fernsteuerung geflogen wird. Ziel ist der Umbau zu einer Plattform, die **ohne GPS in Innenräumen** fliegt, am Boden angebrachte **AprilTags** erkennt und über einem Tag eine **Nutzlast abwirft**.

TEILZIELE

- Autopilot:** Betaflight durch ArduPilot Copter ersetzen, Konfiguration über QGroundControl.
- Position & Altitude Hold** ohne GPS über LiDAR und optischen Fluss.
- Markererkennung** auf dem Companion-Computer, die den **Abwurf** über einen 9g-Servo auslöst.
- Rahmenerweiterung** (3D-Druck) für Sensor, Servo, Pi und Kamera.

Plattform & Hardware

Rahmen	SpeedyBee BEE35 Pro, 3,5" CineWhoop
Flight Controller	Flywoo GOKU GN745 AIO (STM32F745)
Firmware	ArduPilot Copter 4.7.1 — Eigenbau
Antrieb	4 × Emax Eco II 2004, 3000 KV, ESC 45 A
Funkstrecke	ExpressLRS, RadioMaster XR4 Gemini
Companion	Raspberry Pi Zero 2 WH
Kamera	Raspberry Pi AI Camera (Sony IMX500)
Navigation	MicoAir MTF-01P — LiDAR + Optical Flow
Nach vorn	MicoAir MT-15 — Distanz 0,1–15 m
Aktuator	Micro-Servo 9g am Pi (BCM 12)
FPV-Video	RunCam Phoenix 2 → SpeedyBee TX800

Projekt in Zahlen

29 fps Kamerabilder pro Sekunde ausgewertet, im Aufbau gemessen	52 Hz Distanzpakete des vorderen MT-15
0x GPS-Satelliten für die Navigation nötig	1187 gesicherte Flugparameter je Aufzeichnung

Vorgehen

- Bestandsaufnahme & Sicherheitskonzept**
Hardware sichten, Failsafe-Regeln, Prüfliste
- ArduPilot flashen & konfigurieren**
Betaflight ersetzt, Parametersatz gesichert
- MTF-01P in Betrieb nehmen**
UART5, EKF3-Fusion, Telemetrie als CSV verifiziert
- Raspberry Pi anbinden**
UART4-MAVLink, automatisierter Health-Check
- AprilTag-Erkennung auf dem Pi**
tag36h11, zwei Backends, Durchsatz vermessen
- Abwurf durch die Erkennung ausgelöst**
Servo öffnet bei stabil erkanntem Tag
- Flugversuche → Absturz**
21.08.: Totalschaden an der Hallendecke, Aufbau repariert
- Firmware neu gebaut, Sensorik erweitert**
Copter 4.7.1 mit Flussfusion; MT-15 nach vorn in Betrieb
- Erster Flug mit Propellern**
gesperrt: Kompass-Vorflugprüfung schlägt fehl
- Autonome Mission im Flug**
Suchen, Anfliegen, Zentrieren, Abwerfen — offen

Die Drohne



Abb. 1: Die 3,5"-CineWhoop im Flug — ducted Propeller und Propellerschutz erlauben Flüge in Innenräumen und in Personennähe.

Systemarchitektur

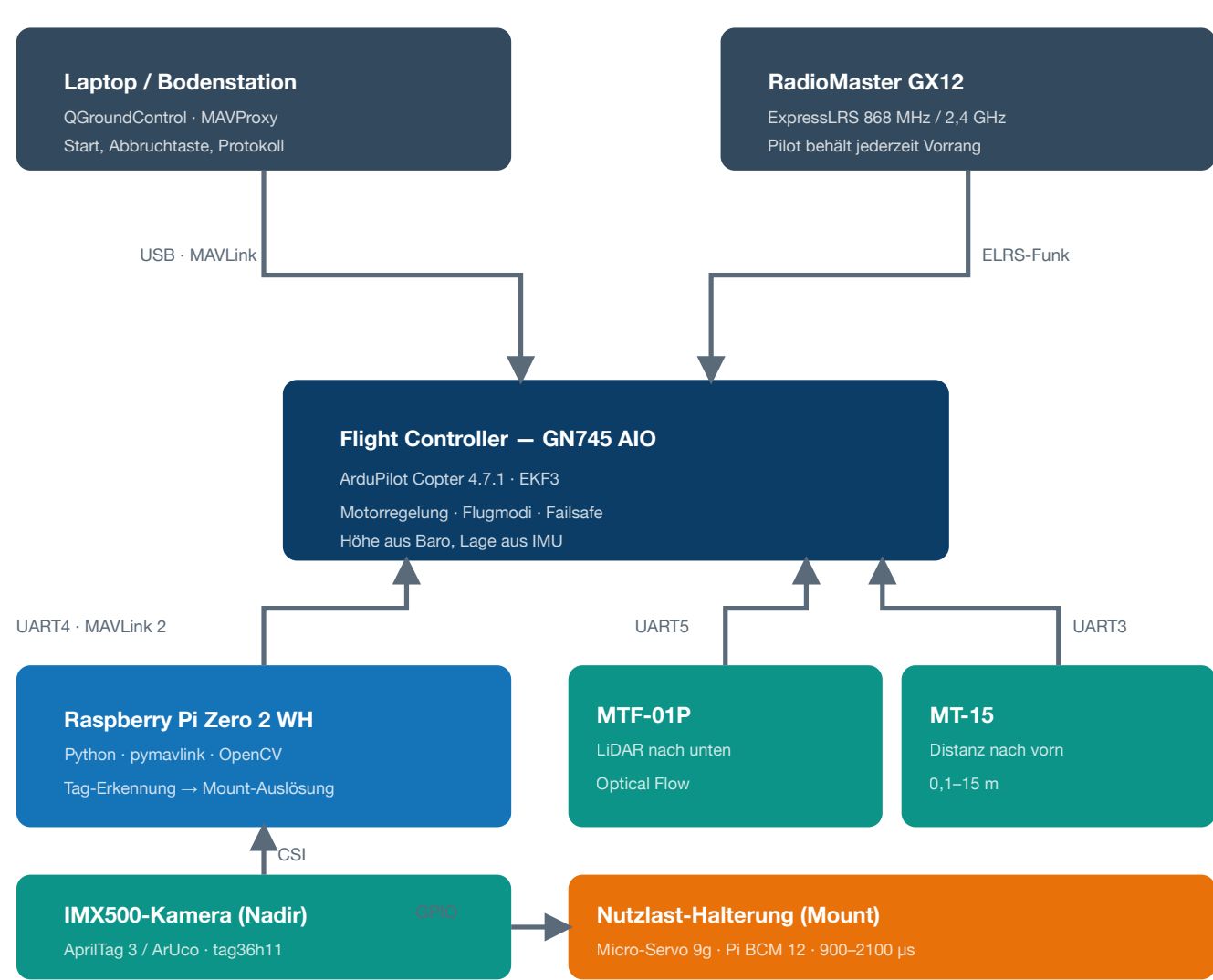


Abb. 2: Signalfluss zwischen Bodenstation, Autopilot, Companion-Computer und Sensorik.

Fliegen ohne GPS

In Innenräumen fehlt das GPS-Signal. Der **MTF-01P** ersetzt es durch zwei Messgrößen, die der EKF3-Zustandsschätzer von ArduPilot direkt verarbeitet:

- LiDAR** misst den Abstand zum Boden (1–800 cm) → Höhe über Grund.
- Optischer Fluss** schätzt aus der Bildverschiebung die horizontale Geschwindigkeit → Halten der Position ohne Satelliten.

Dass das lange nicht funktionierte, lag nicht an der Konfiguration: Im installierten Firmware-Image war die Flussfusion des EKF3 **gar nicht einkompiliert** — kein Parameter hätte daran etwas geändert. Zusätzlich verlangte `EK3_SRC1_POSXY=3` eine GPS-Position bei abgeschaltetem GPS. Die Konsequenz ist ein eigener Build, **ArduCopter 4.7.1** mit Flussfusion und `GUIDED_NOGPS`. Seitdem meldet der EKF beim Anheben eine relative Horizontalposition — vorher nie.

Zwischenergebnis: Tag erkannt → Last fällt

Die Kette Kamera → Erkennung → Aktuator funktioniert : Die nach unten gerichtete IMX500 erkennt <code>tag36h11</code> -Marker, und bei Tag ID 3 öffnet der Servo die Nutzlast-Halterung.
Zwei Detektor-Backends: AprilTag 3 nativ (meldet Hamming-Korrektur und Decision Margin) und OpenCV <code>ArucoDetector</code> als Rückfall und für die Kalibrierung. Durchsatz auf dem Pi Zero 2 W: 29 fps bei 640×480 inklusive Bildaufnahme; im 30-Sekunden-Lauf am Aufbau 877 ausgewertete Bilder. Auslösung: Drei aufeinanderfolgende frische Erkennungen ohne Hamming-Korrektur und mit Decision Margin ≥ 30. Der Servo hängt direkt am Pi (BCM 12), und die Halterung öffnet genau einmal pro Lauf. Auch im Flug benutzbar: am Boden, vor dem Armieren oder während eines manuell geflogenen Fluges — die Aufzeichnung sendet selbst keinen Armier-, Modus- oder Motorbefehl. Ohne Kalibrierung keine Entfernung: Die Software meldet dann bewusst nur IDs und Eckpunkte statt einer erfundenen Distanz.
<i>Verifiziert am Boden und handgeführt — nicht im autonomen Flug.</i>

Rahmen & 3D-Druck

Die Rahmenerweiterung trägt Sensor, Servo, Pi und Kamera. Jedes Teil liegt zweifach im Repository: als **STL** (Geometrie) und als **3MF** mit den Einstellungen, mit denen gedruckt wurde.

- Pi-Gehäuse** in PETG, 37,6 × 72,6 × 13,4 mm, Stützen sind Teil des Modells statt vom Slicer erzeugt.
- Weiche Rahmenteile** (vorn, hinten, 2 × seitlich) in TPU 95A. Sie dämpfen Schwingungen, und genau davon hängen optischer Fluss und die Zustandsschätzung ab — starres Filament ist kein Ersatz.
- Servohalter** für den 9g-Servo, Stellbereich 900–2100 µs, Neutral 1500 µs; Anlaufstrom bis 1,6 A über den geregelten 5-V-Zweig.
- Prusa XL, 0,4-mm-Düse, 0,20 mm Schichthöhe, 30 % Infill; von den weichen Teilen gibt es inzwischen eine überarbeitete zweite Fassung.

Software & Werkzeuge

Autopilot	ArduPilot Copter · QGroundControl
Protokoll	MAVLink 2 (pymavlink, MAVProxy)
Companion	Raspberry Pi OS · Python 3.13 · uv
Erkennung	AprilTag 3 · OpenCV ArUco (36h11)
Werkzeuge	drone-check · -inspect · -walk · -mount
Qualität	ruff · ty · pytest · import-linter

Flugversuche

Der vollständig autonome Flug wurde **nicht erreicht**. Die Versuche vom 20./21. August sind trotzdem das lehrreichste Ergebnis — jeder ist über Telemetrie und Dataflash-Log ausgewertet.

20.08.	Erster Flug überhaupt. Stieg über die befohlene Höhe hinaus und blieb oben; gestoppt durch Abziehen des Akkus, die Drohne hing an einer Leine.
21.08. vormittags	Zwei STABILIZE-Starts, kein Abheben. Der Rangefinder meldet über 13 s konstant 0,02 m. Der befohlene Schub lag bei 0,128 gegen einen gelernten Hover von 0,263.
21.08. Totalschaden	Der Abbruch flog die Drohne. Der Timeout löste eine Landung aus — und LAND ist höhengeregelt. In einem einzigen Log-Sample ging der Schub von 0,128 auf 1,000 und die Drohne stieg in einer Sekunde in die Hallendecke.
21.08. abends	Erstes echtes Abheben nach dem Wiederaufbau: 0,02 m → 0,57 m in 0,4 s. Der Höhenwächter stoppte den Steigflug korrekt.
21.08. abends	Wächter greift. „Steigt mit +0,96 m/s“ meldet das Fahrzeug, während der Rangefinder 0,00 m/s misst — Abbruch nach 0,8 s Widerspruch, noch bevor Schub aufgebaut wurde.

Drei Ursachen, gemessen

- Der EKF hatte keine Höhenquelle.** `EK3_SRC1_POSZ=2` benannte den Rangefinder, `EK3_RNG_USE_HGT=-1` verbot dessen Nutzung. Der Filter meldete **−10 000 m** und **−38 m/s Sinken**, während die Drohne auf dem Boden stand. STABILIZE ignoriert das; LAND regelt danach. Mit dem Barometer als Quelle: +0,05 m und +0,01 m/s.
- Die Schubkennlinie war falsch angenommen.** ArduPilot formt den STABILIZE-Stick kubisch, sodass *Stickmitte* Hover-Schub ist. Der Code setzte den Stick auf 0,313 im Glauben, das sei Hover — die Kennlinie macht daraus 0,135, also etwa die halbe nötige Kraft.
- Ein Beschleunigungs-Offset im Betrieb.** Stillstehend misst die Drohne 9,79 m/s²; mit laufenden Motoren im Leerlauf 10,65 m/s² bei unveränderter Lage. EKF3 integriert die Differenz zu 0,75 m/s² Drift pro Sekunde — daher „Steigt mit 4,27 m/s“ bei 0,020 m Bodenabstand. Die Vibrationsmetrik bleibt dabei blind, weil sie Streuung misst und ein gleichgerichteter Rüttler ein Gleichanteil ist.

Video: der Absturz

	Aufnahme vom 21.08.2026 Der zweite STABILIZE-Start: 13 Sekunden ohne Bewegung, dann der Abbruch, der die Drohne in einem Zug in die Hallendecke fliegt. <code>photos.app.goo.gl/q4YqJamv6uWtpWg16</code>
--	---

Abb. 3: Der Absturz, gefilmt vom Boden — QR-Code scannen.

Absicherung vor dem Flug

Geprobt wird gegen **echte ArduPilot-SITL**, gebaut aus genau dem Commit, der auch auf der Drohne liegt. Drei Szenarien laufen durch: Schweben auf optischem Fluss, Koexistenz beider Abstandssensoren, und Verlust der Bodenstation mit Landung und Entwaffnung. Dazu über **800 Offline-Tests**.

Warum nicht mehr der eigene Simulator: Der enthielt dieselbe falsche Annahme über die Schubkennlinie wie der Flugcode und konnte den Fehler deshalb prinzipiell nicht finden. Eine Simulation prüft Code gegen Annahmen — nicht Annahmen gegen die Wirklichkeit.

Konsequenzen im Code

- Landen ist keine Notbremse mehr.** Steht die Drohne noch am Boden, wird entwaffnet statt gelandet.
- Ein Flug gilt erst als begonnen, wenn die Drohne **abgehoben ist** — vorher ist ein Fehler kein Notfall in der Luft.
- Vor dem Armieren muss die Vertikalschätzung **3 s ruhig** sein; Steigrate und Rangefinder werden laufend gegeneinander geprüft.
- Jeder Abbau prüft den Disarm und erzwingt ihn am Boden; bricht die Verbindung ab, beendet das den Flug geordnet.
- Vorflugprüfungen nie abschalten. Copter 4.7 dreht die Logik um: `ARMING_SKIPCHK=0` heißt, dass keine Prüfung übersprungen wird — und geprüft werden Werte, nicht nur Statusbits.

Stand & Ausblick

- Erreicht:** eigene Firmware mit Flussfusion, Sensorik nach unten und nach vorn in Betrieb, AprilTag-Erkennung und die davon ausgelöste Nutzlast-Freigabe.
- Der Blocker:** `PreArm: Check mag field` — das Z-Feld weicht um rund **295 mG** gegen eine 200-mG-Schwelle ab. Die Drohne ist damit **nicht für Tests mit Propellern freigegeben**. Die Prüfung wird nicht umgangen, sondern die Ursache gesucht.
- Offen:** der erste Flug unter der neuen Firmware und die autonome Mission. Und eine Grenze bleibt: ein Abstandswert nach vorn ist noch keine Hindernisvermeidung — die Firmware bringt keine Proximity-Unterstützung mit.